

NS-3



web-platform <https://ns3.netsimsolutions.com>

Step 1: create yourself account

▼ Web-platform overview



Why ns-3 Uses C++ (and not Python)

Despite Python being easier to learn and write, **simulators like ns-3 still use C++** for very solid, performance-driven reasons.

1. Performance Is Critical

- **Network simulators** like ns-3 model very low-level events (packet transmission, queue delays, routing logic).
- C++ is **faster** than Python — it's compiled to machine code, while Python is interpreted.
- When you simulate thousands of packets or nodes, you can't afford Python's speed penalty.

2. Memory Control

- C++ gives you **fine-grained control over memory and data structures** (important when simulating hardware-like behavior or time-critical systems).
- Python abstracts too much — great for usability, bad for simulation accuracy or scalability.

▼ "Catalog" Tile

the **"Catalog"** tile refers to a curated collection of **pre-built simulation projects or example scripts** that you can:

- **Browse**
- **Launch** directly in the web-based interface
- **Modify and run** interactively without installing ns-3 locally

Select the ns-3 code base from the list below:

Index	Name	Repository	Selection
1	ns-3.44	https://gitlab.com/nsnam/ns-3-dev/-/tree/ns-3.44	☐
2	5g-nr	https://gitlab.com/cttc-lena/nr/-/tree/5g-lena-v4.0.y	☒
3	nr v2x	https://gitlab.com/cttc-lena/nr/-/tree/v2x-1.1	☐
4	lorawan	https://github.com/signetlabdei/lorawan/tree/v0.3.2	☐
5	ns3-oran	https://github.com/usnistgov/ns3-oran/tree/v1.3	☐
6	sns3-satellite	https://github.com/sns3/sns3-satellite/tree/3.43	☐

Previous Next

▼ "Dashboard" Tile

Start
Dashboard
Catalog
Customization
Subscription
JupyterLab
Help

Powered by NetSimSolutions LLC &




Download All Files
Delete All Simulations
Stop All Simulations

Scenario	Name	Status	Progress
simple-point-to-point-olsr	point2point	draft	0% ...
adaptive-red-tests	sim1	completed	100% ...
wifi-power-adaptation-distance	wifi_power_sim_1	completed	100% ...

Previous
Next

Crate a new [simulation](#)

▼ Key Abstract Class

1. **Node** - In *ns-3* the basic computing device abstraction is called the node. This abstraction is represented in C++ by the class `Node`. The `Node` class provides methods for managing the representations of computing devices in simulations.



Any device you think Node could be?

2. **Application** - Just as software applications run on computers to perform tasks in the "real world," *ns-3* applications run on *ns-3* `Nodes` to drive simulations in the simulated world.
3. **Channel** - When you connect your Ethernet cable to the plug in the wall, you are connecting your computer to an Ethernet communication channel. In the simulated world of *ns-3*, one connects a `Node` to an object representing a communication channel.

The `Channel` class provides methods for **managing communication subnetwork** objects and connecting nodes to them.

`Channels` may also be specialized by developers, A `Channel` specialization may model something as simple as a wire. The specialized `Channel` can also model things as complicated as a large Ethernet switch, or three-dimensional space full of obstructions in the case of wireless networks (wireless scenario)

4. **Net Device** - if you wanted to connect a computer to a network, you had to buy a specific kind of network cable and a hardware device called Network Interface Cards, or *NICs*.

In *ns-3* the *net device* abstraction covers both the software driver and the simulated hardware. A net device is "installed" in a `Node` in order to enable the `Node` to communicate with other `Nodes` in the simulation via `Channels`. Just as in a real computer, a `Node` may be connected to more than one `Channel` via multiple `NetDevices`.

5. **Topology** - In a **large simulated network** you will need to arrange many connections between `Nodes`, `NetDevices` and `Channels`. Since connecting `NetDevices` to `Nodes`, `NetDevices` to `Channels`, assigning IP addresses, etc., are such common tasks in *ns-3*, *ns-3* provide what we call *topologyhelpers* those many distinct operations into an easy to use model for your convenience.

▼ NS-3 scripts

1. **Module Includes:** In order to deal with the large number of include files present in the system, we group includes according to relatively large modules.

```
#include"ns3/core-module.h"
#include"ns3/network-module.h"
#include"ns3/internet-module.h"
#include"ns3/point-to-point-module.h"
#include"ns3/applications-module.h"
```

2. **Ns3 Namespace:** The *ns-3* project is implemented in a C++ namespace called `ns3`.

```
using namespace ns3;
```

3. **Main Function:** Just as in any C++ program, you need to define a main function that will be the first function run.

```
int main(int argc, char *argv[])
{}
```

4. Logging

```
NS_LOG_COMPONENT_DEFINE("xxxx");
```

▼ Object Flow:

```
[main()]
|— NodeContainer nodes
|   |— Create(2) → Node 0, Node 1
|
|— PointToPointHelper
|   |— Install(nodes) → NetDevices on Node 0 ↔ Node 1
|
|— InternetStackHelper
```

```

|   └─ Install(nodes) → Adds IP, routing
|
|   └─ Ipv4AddressHelper
|       └─ Assign(devices) → IP: 10.1.1.1, 10.1.1.2
|
|   └─ UdpEchoServerHelper(9)
|       └─ Install(Node 1) → Server app listens on port 9
|
|   └─ UdpEchoClientHelper(10.1.1.2, 9)
|       └─ Install(Node 0) → Client sends packet to server
|
|   └─ Simulator::Run()
|   └─ Simulator::Destroy()

```

▼ Topology Helpers

- **(Create Nodes) NodeContainer:** create the *ns-3* `Node` objects that will represent the computers in the simulation.

```

// calls the Create method on the nodes object and asks the container to create two nodes.
NodeContainer nodes;
nodes.Create(2);

```

- **(Setup Links) PointToPointHelper:** is used to configure and connect *ns-3* `PointToPointNetDevice` and `PointToPointChannel` objects

```

PointToPointHelper pointToPoint;
pointToPoint.SetDeviceAttribute("DataRate", StringValue("5Mbps"));
pointToPoint.SetChannelAttribute("Delay", StringValue("2ms"));

```

- **(install Device) NetDeviceContainer**

ready to make `PointToPointNetDevices` and wire `PointToPointChannel` objects between them. So configuring the devices and channel:

```
NetDeviceContainer devices;  
devices = pointToPoint.Install(nodes);
```

- **(Install Internet Stack) InternetStackHelper**

We now have nodes and devices configured, but we don't have any protocol stacks installed on our nodes. The next two lines of code will take care of that.

```
InternetStackHelper stack;  
stack.Install(nodes);
```

- **(Assign IP Addresses) Ipv4AddressHelper**

```
Ipv4AddressHelper address;  
address.SetBase("10.1.1.0", "255.255.255.0");  
  
Ipv4InterfaceContainer interfaces = address.Assign(devices);
```

declare an address helper object and tell it that it should begin allocating IP addresses from the network 10.1.1.0 using the mask 255.255.255.0 to define the allocatable bits

▼ Application Helpers **(Install Applications)**

- **UdpEchoServerHelper:** creating a **UDP Echo Server**

```
UdpEchoServerHelper echoServer(9); // Equivalent to "listen for U  
DP packets on port 9"  
  
// make the underlying echo server application to be instantiated an  
d attached to a node.  
ApplicationContainer serverApps = echoServer.Install(nodes.Get  
(1));  
serverApps.Start(Seconds(1));  
serverApps.Stop(Seconds(10));
```

- **UdpEchoClientHelper:**

```
// this client will send packets to the echo server you installed on th
at IP address
UdpEchoClientHelper echoClient(interfaces.GetAddress(1), 9);
// Configure client behavior
echoClient.SetAttribute("MaxPackets", UIntegerValue(1));
echoClient.SetAttribute("Interval", TimeValue(Seconds(1)));
echoClient.SetAttribute("PacketSize", UIntegerValue(1024));

ApplicationContainer clientApps = echoClient.Install(nodes.Get(0));
clientApps.Start(Seconds(2));
clientApps.Stop(Seconds(10));
```

▼ Simulator:

run the simulation using the global function

```
Simulator::Run();
```

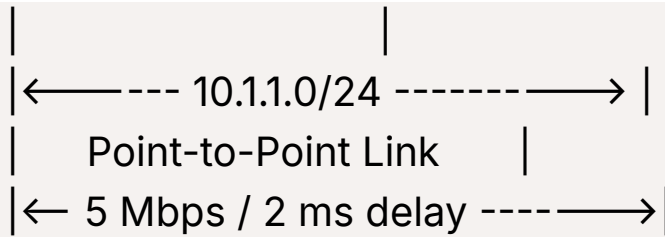
When the simulator will stop?

```
Simulator::Destroy();
return 0;
}
```

Summary: This's a **simple UDP echo over a point-to-point link**.

Network Topology:

Node 0 (Client)	Node 1 (Server)
--------------------	--------------------



- **UDP packet from client** goes from Node 0 → Node 1 over point-to-point link.
- **Server replies** back to client with the same packet.

▼ Full script:

```

/*
 * SPDX-License-Identifier: GPL-2.0-only
 */

#include "ns3/applications-module.h"
#include "ns3/core-module.h"
#include "ns3/internet-module.h"
#include "ns3/network-module.h"
#include "ns3/point-to-point-module.h"

// Default Network Topology
//
// 10.1.1.0
// n0 ----- n1
// point-to-point
//

using namespace ns3;

NS_LOG_COMPONENT_DEFINE("FirstScriptExample");

int

```



```

main(int argc, char* argv[])
{
    CommandLine cmd(__FILE__);
    cmd.Parse(argc, argv);

    // sets the time resolution to one nanosecond
    Time::SetResolution(Time::NS);
    LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_INFO);
    LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_INFO)

    NodeContainer nodes;
    nodes.Create(2);

    PointToPointHelper pointToPoint;
    pointToPoint.SetDeviceAttribute("DataRate", StringValue("5Mbps"));
    pointToPoint.SetChannelAttribute("Delay", StringValue("2ms"));

    NetDeviceContainer devices;
    devices = pointToPoint.Install(nodes);

    InternetStackHelper stack;
    stack.Install(nodes);

    Ipv4AddressHelper address;
    address.SetBase("10.1.1.0", "255.255.255.0");

    Ipv4InterfaceContainer interfaces = address.Assign(devices);

    UdpEchoServerHelper echoServer(9);

    ApplicationContainer serverApps = echoServer.Install(nodes.Get(1));
    serverApps.Start(Seconds(1));
    serverApps.Stop(Seconds(10));

    UdpEchoClientHelper echoClient(interfaces.GetAddress(1), 9);
    echoClient.SetAttribute("MaxPackets", UIntegerValue(1));

```

```

echoClient.SetAttribute("Interval", TimeValue(Seconds(1)));
echoClient.SetAttribute("PacketSize", UIntegerValue(1024));

ApplicationContainer clientApps = echoClient.Install(nodes.Get(0));
clientApps.Start(Seconds(2));
clientApps.Stop(Seconds(10));

Simulator::Run();
Simulator::Destroy();
return 0;
}

```

Terminal Output for first : first

```

$ ./ns3 build
[ 0%] Building CXX object
scratch/CMakeFiles/scratch_zhijinns3.first.first_netsimllc_zhijinns3_first_first.dir/zhijinns3.first.first/netsimllc_zhijinns3_first_first.cc.o
[ 0%] Linking CXX executable /home/ubuntu/ns3-repo/2/debug/build/scratch/zhijinns3.first.first/ns3.44-netsimllc_zhijinns3_first_first-debug
At time +2s client sent 1024 bytes to 10.1.1.2 port 9
At time +2.00369s server received 1024 bytes from 10.1.1.1 port 49153
At time +2.00369s server sent 1024 bytes to 10.1.1.1 port 49153
At time +2.00737s client received 1024 bytes from 10.1.1.2 port 9
$ ./ns3 run xxx/first

```



(practice) Test/Modify Suggestions

You can try:

- Increasing `MaxPackets` to 5
- Changing `PacketSize`
- Adjusting `DataRate` and `Delay`
- Adding a second point-to-point link with routing

▼ simple-point-to-point-olsr

sets up a network with Multi-node topology with the **OLSR (Optimized Link State Routing)** protocol to manage routing.



High-Level Purpose

This script sets up a simple network of 3 nodes connected via point-to-point links, **installs the OLSR routing protocol**, and demonstrates packet flow using UDP echo applications.

◆ Routing == Knowing How to Forward Packets

Routing protocols (like OLSR) tell nodes:

"If you want to reach IP X.X.X.X, send it through interface Y to neighbor Z."

Key differences:

- Installs IP stack and sets OLSR as the routing protocol.

```
OlsrHelper olsr;  
InternetStackHelper stack;  
stack.SetRoutingHelper(olsr); // Use OLSR for routing  
stack.Install(nodes);
```

- 3 Node topology:

```
// Creates three nodes: Node 0, Node 1, and Node 2  
NodeContainer nodes;  
nodes.Create(3);  
  
...  
  
// Set Up Links  
PointToPointHelper pointToPoint;  
pointToPoint.SetDeviceAttribute("DataRate", StringValue("5Mbps"));  
pointToPoint.SetChannelAttribute("Delay", StringValue("2ms"));  
// Node0 --- Node1 --- Node2  
NetDeviceContainer devices01 = pointToPoint.Install(nodes.Get(0), nodes  
NetDeviceContainer devices12 = pointToPoint.Install(nodes.Get(1), nodes.
```

...

```
// Assigns IPs to the two links:  
// Node0 ↔ Node1 uses 10.1.1.0/24  
// Node1 ↔ Node2 uses 10.1.2.0/24  
Ipv4AddressHelper address;  
address.SetBase("10.1.1.0", "255.255.255.0");  
Ipv4InterfaceContainer interfaces01 = address.Assign(devices01);  
  
address.SetBase("10.1.2.0", "255.255.255.0");  
Ipv4InterfaceContainer interfaces12 = address.Assign(devices12);
```

...

Aspect	3-nodes OLSR Routing	2-node UDP Echo
Number of Nodes	3 or more (n0–n1–n2...)	2 (n0–n1)
Topology	Linear (or extended) multi-hop	Simple direct link
Routing	Uses OLSR (dynamic proactive routing)	No routing protocol needed
Link Type	Point-to-point	Point-to-point
Use of Ipv4RoutingHelper	Yes (via OlsrHelper)	No
Client–Server Path	Indirect (may require hop through n1)	Direct (n0 ↔ n1)
Protocol Setup Complexity	Higher (must configure routing)	Minimal (direct IP)
Simulation Purpose	Tests routing over a multi-hop topology	Tests simple UDP send/recv
IP Configuration	Multiple subnets (10.1.1.0/24, 10.1.2.0/24, etc.)	Single subnet (10.1.1.0/24)
Routing Table Computation	Automatic via OLSR	Not required

Packet Traversal	Client → Router → Server	Client → Server
Educational Value	Shows how routing protocols forward packets	Shows basic packet send/receive with delays